THE ANDROID STACK

aitrich TECHNOLOGIES

aTP
aitrich trained professional

INTRODUCTION

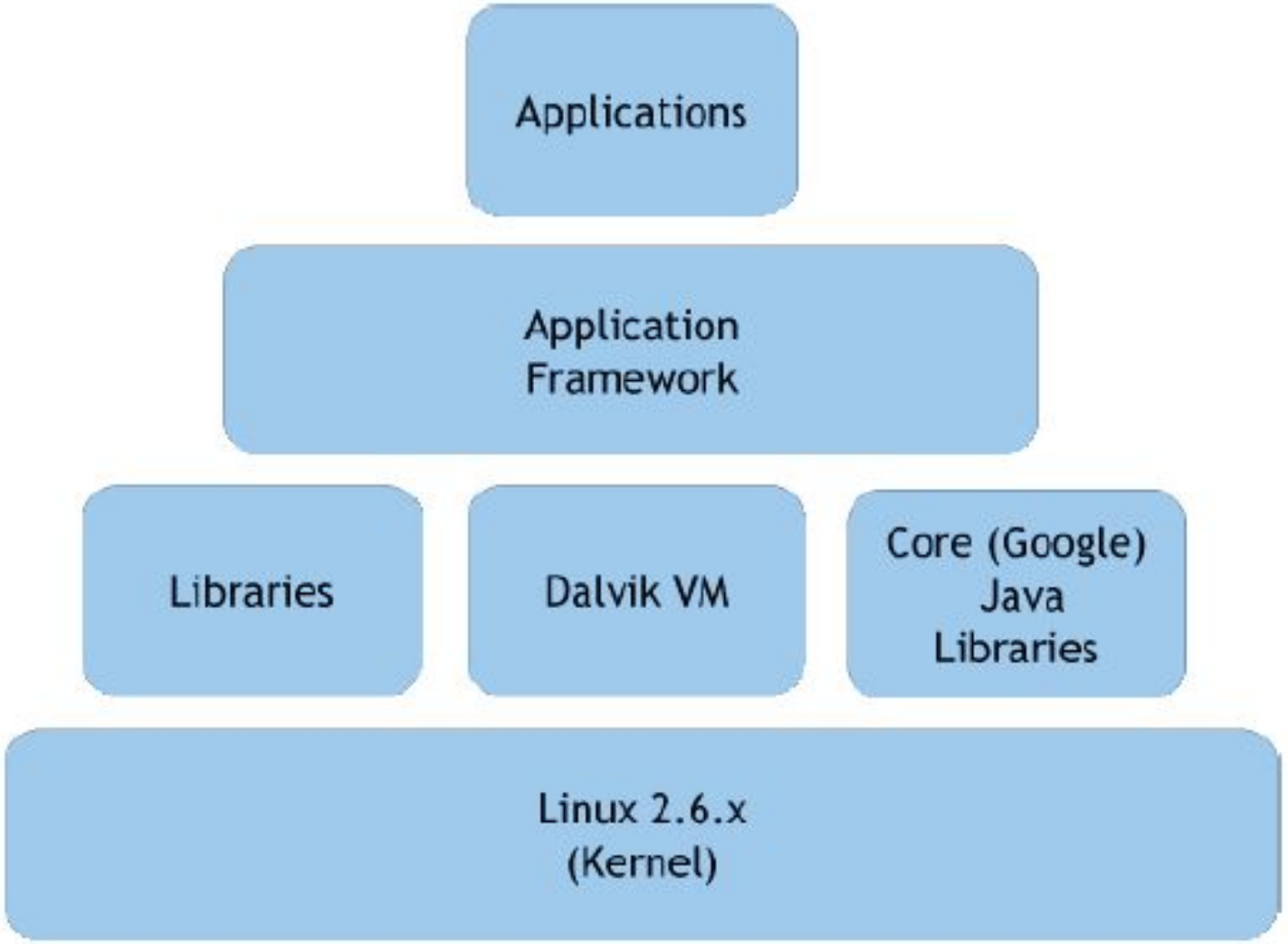
- Introduction
- Android Stack
- Linux kernel
- Native Libraries
- Android Runtime
- Application Framework
- Applications
- Platform Start-up

INTRODUCTION

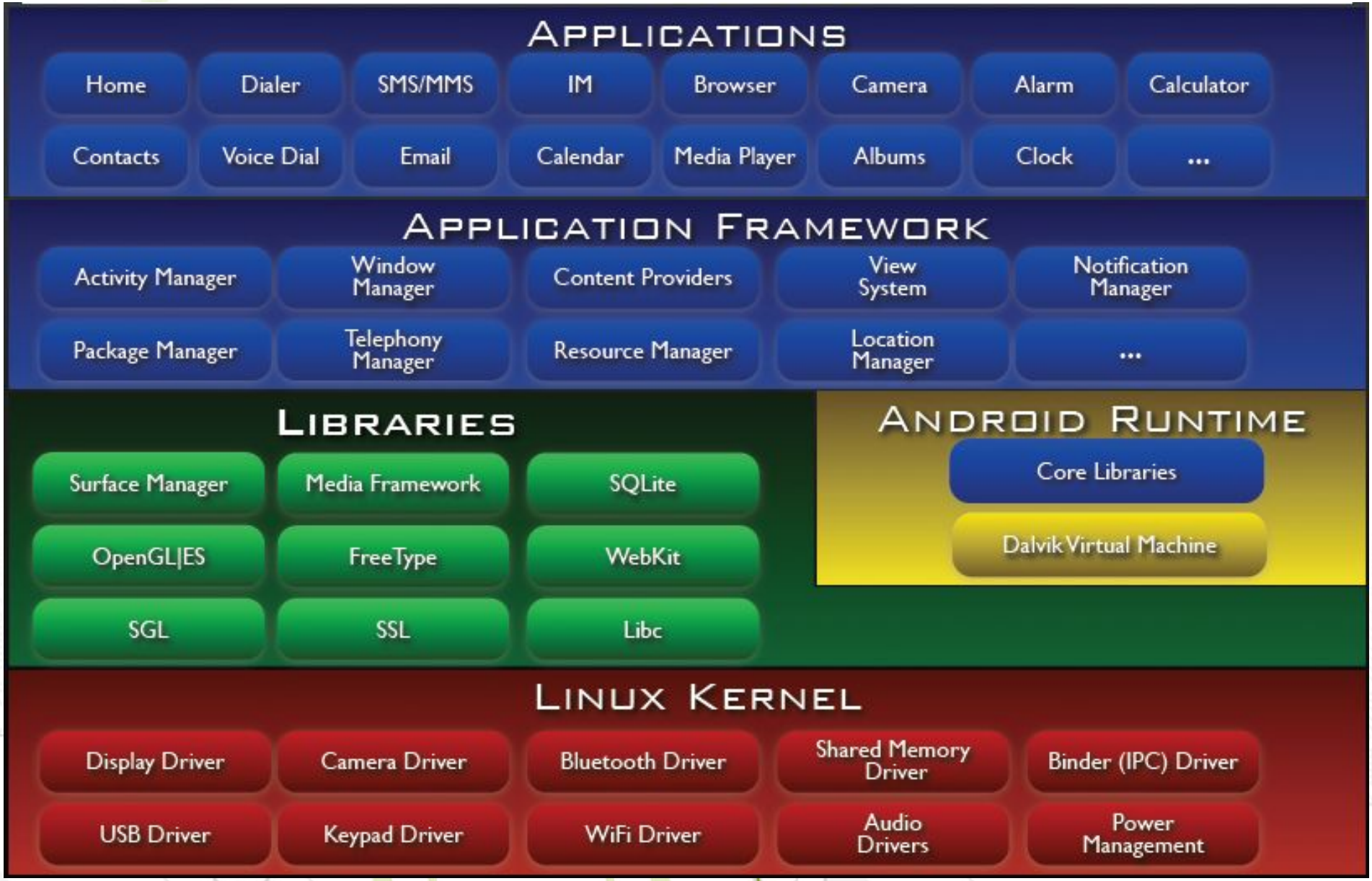
- Google defines Android as a "software stack" for mobile phones that includes the operating system (the platform on which everything runs) the middleware (the programming that allows applications to talk to a network and to one another), and the applications (the actual programs that the phones will run).

- Android is based on the Linux operating system, and all of its applications will be written using Java.
- Android "ship with a set of core applications including an email client, SMS program, calendar, maps, browser, contacts," and more.

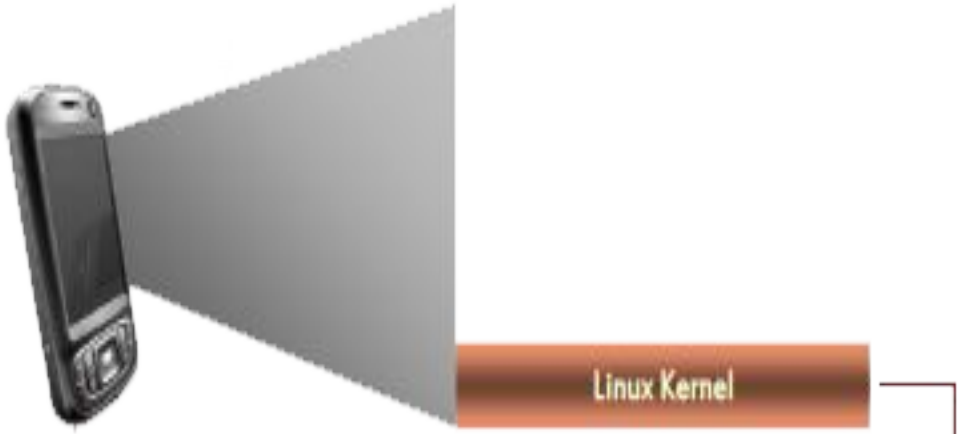
Android Software Stack

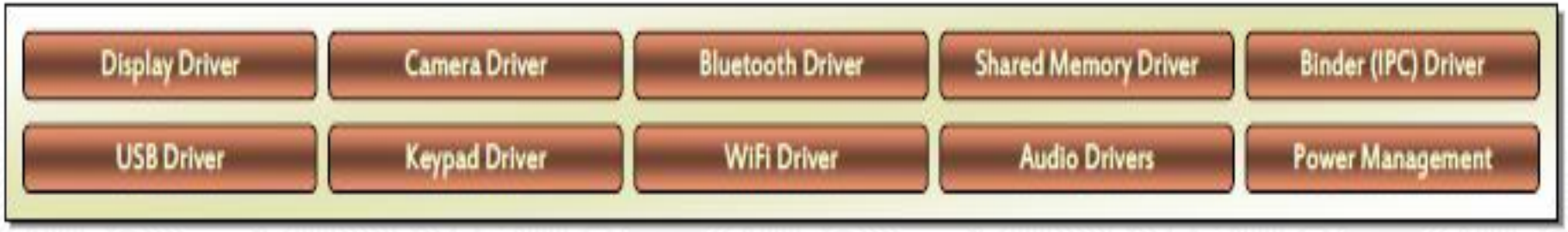


Android Architecture



Linux Kernel





Linux Kernel

- Android is built on the standard Linux 2.6 kernel , but Android is not Linux
- Does not include the full set of standard Linux utilities, Patch of “kernel enhancements” to support Android.
- Acts as a hardware abstraction layer between the physical hardware of the handset and the Android software stack.

Linux Kernel

Why Linux Kernel?

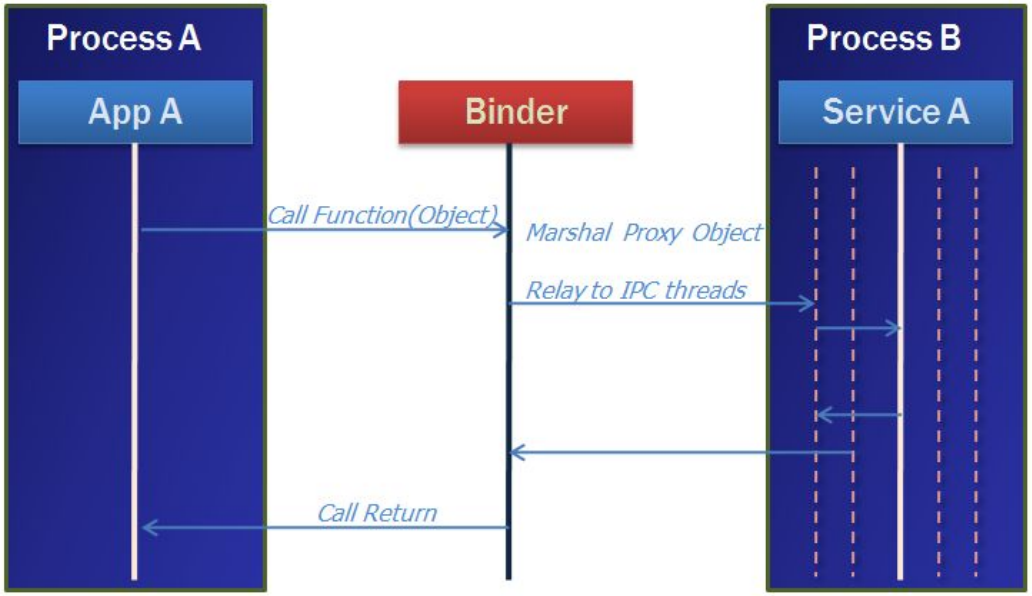
- Great memory and process management
- Permissions-based security model
- Proven driver model
- Support for shared libraries
- It's already open source!

Linux Kernel - Binder

- An OpenBinder-based driver to facilitate inter-process communication (IPC) on the Android platform.
- Driver to facilitate IPC between applications and services
- OpenBinder provides:
  - High performance through shared memory.
  - Per-process thread pool for processing requests.
  - Synchronous calls between processes.

Linux Kernel - Binder

Binder in Action



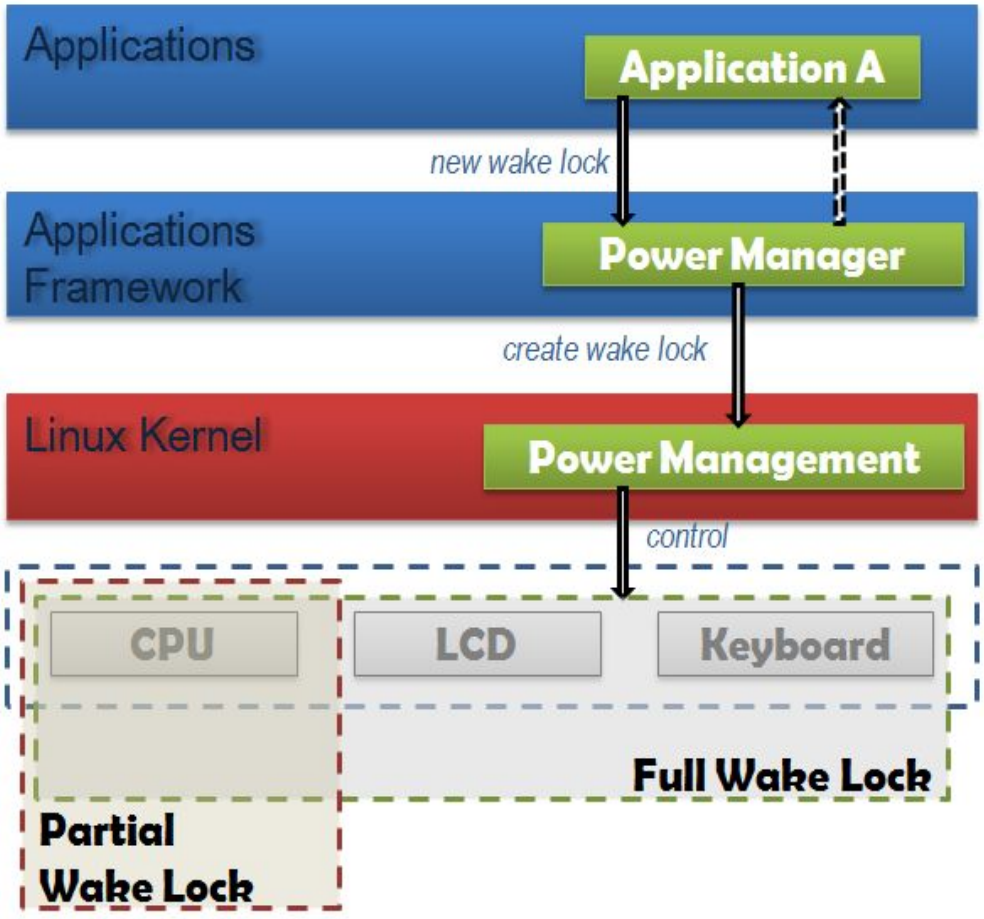
- A pool of threads is associated to each service application to process incoming IPC.
- The driver(Binder) performs mapping of object between two processes.
- Binder uses an object reference as an address in a process's memory space.

Linux Kernel – Power Management

- A light-weight power management driver built atop the standard Linux power management
- Constrained power management policy:
  - CPU shouldn't consume power if no applications or services require power.
  - Components make requests to keep the power on through "*Wake Locks*".

Linux Kernel - Power Management

Power Management in Action

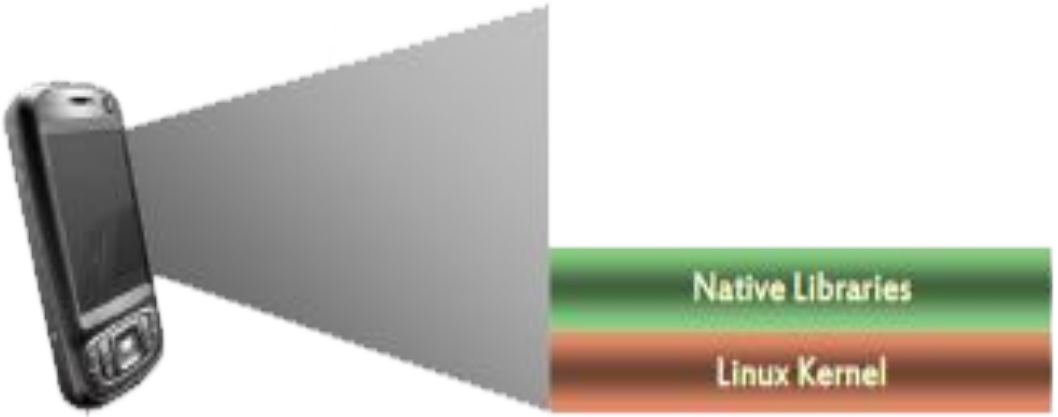


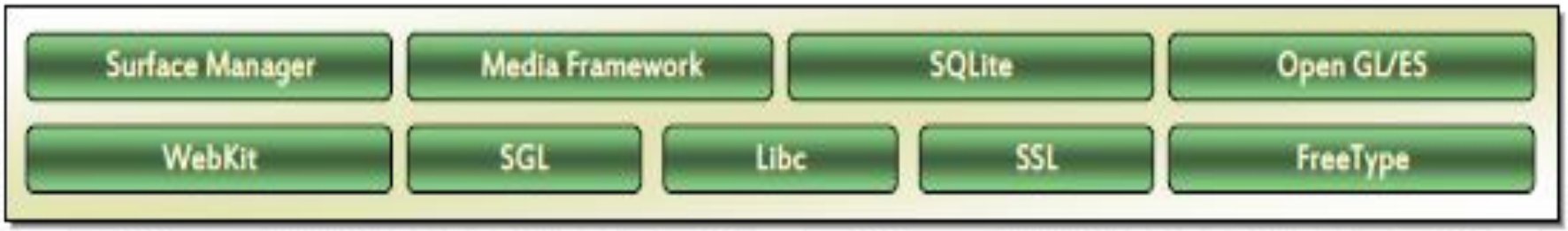
- If there are no active wake locks, CPU will be turned off.
- If there are no partial wake locks, screen and keyboard will be turned off.

Linux Kernel – Other Components

- **Logger**: A lightweight logging device used to capture system, radio, logdata, etc.
- **Android Alarm**: A driver which provides timers that can wake the device up from sleep and a monotonic timebase that runs while the device is asleep
- **M-Systems**: for memory cards and other ash media storage.
- Other conventional drivers:
  - Bluetooth, Display, Keypad, Audio, Etc…

Native Libraries





Native Libraries

- Libraries are set of instructions that tells the device how to handle different kinds of data.
- for example media framework library that supports playback and recording of various audio, video and picture formats.
- Android Native Libraries are written in C/C++ internally, but you'll be calling them through Java interfaces.
- These capabilities are exposed to developers through the android application framework.

Native Libraries - Bionic Libc

- What is bionic?
  - Custom libc (standard C system library) implementation, optimized for embedded use.
- Why build a custom libc library?
  - Size: will load in each process, so it needs to be small
  - Fast: limited CPU power means we need to be fast
- All native code must be compiled against bionic, not glibc (the GNU version of libc).

Native Libraries – LibWebCore (WebKit)

- A framework providing the basis for building a web browser based on open source WebKit browser.
- Properties
  - Ability to render pages in full (desktop) view
  - Full CSS, JavaScript, DOM, AJAX support
  - Support for single-column and adaptive view rendering

Native Libraries – Media Framework

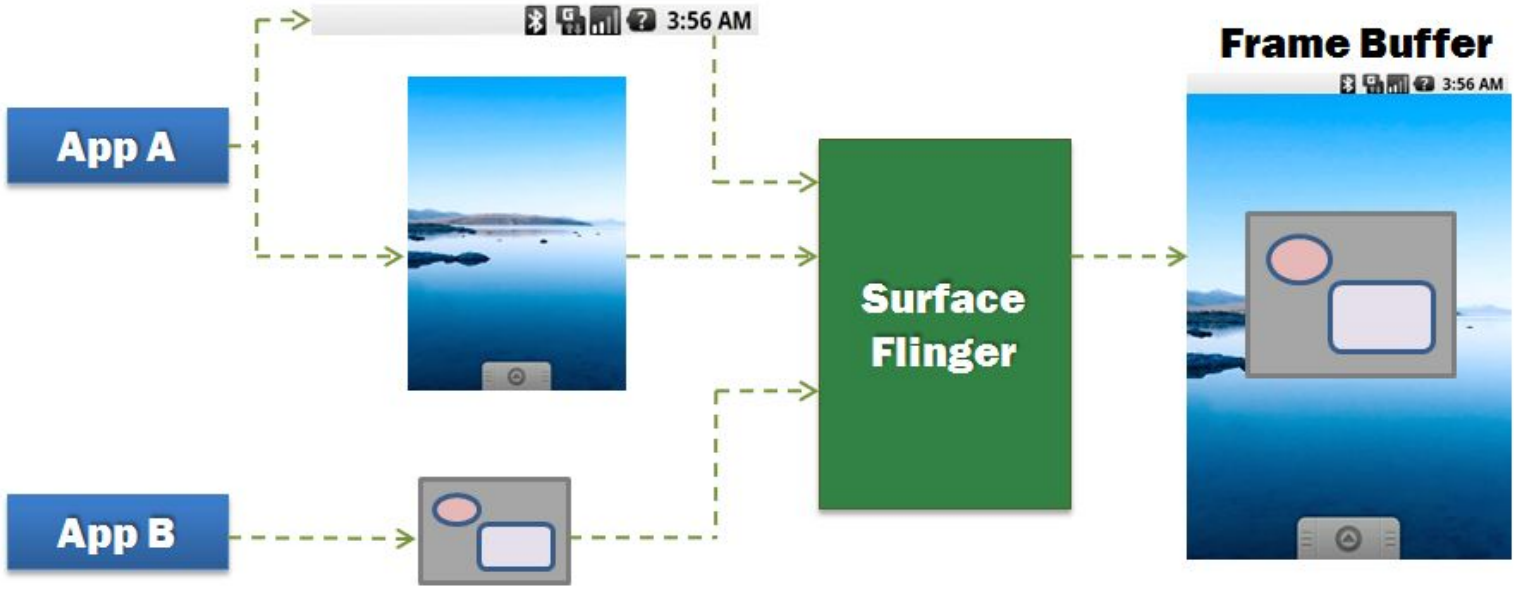
- A media framework based on PacketVideo OpenCore platform.
- The OpenCore provides a universal structure for mobile multimedia applications
- Support for standard video, audio, still-frame formats.
- Support for hardware/software codec plug-ins.

Native Libraries - SQLite

- A powerful lightweight embedded relational database management system contained in a relatively small (500 kb) C library.
- The engine is not a standalone entity. Instead, a SQLite database is just another program linked in and thus becomes an integral part of the program.
- The database is stored as a single cross-platform file on the host machine
- Application access SQLite through conventional simple function calls

Native Libraries- Surface Manager (Surface Flinger)

- A Surface manager to provide display management.
  - Provides system-wide surface “composer”, handling all surface rendering to frame buffer device.
  - Can combine 2D and 3D surfaces and surfaces from multiple applications.



Native Libraries- Audio Manager (Audio Flinger)

- Manages all audio output devices
- Processes multiple audio streams into PCM audio out paths
- Handles audio routing to various outputs

Native Libraries – OpenGL|ES & SGL

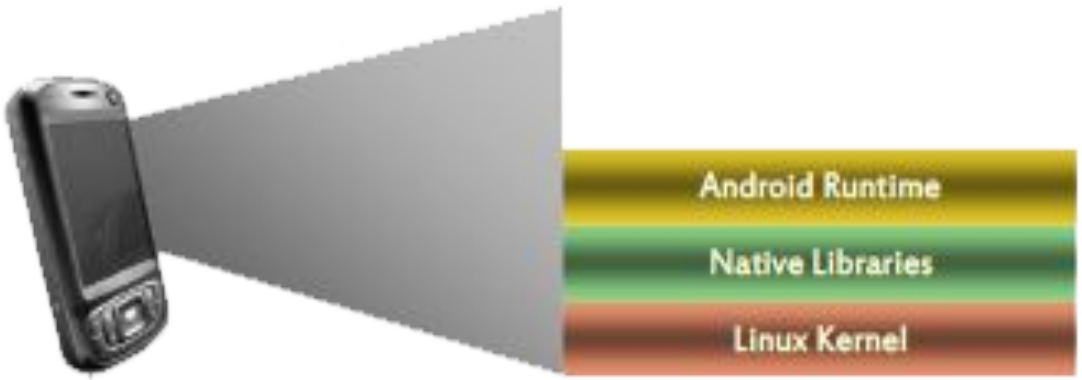
- **OpenGL|ES**
  - 3D Library
  - Using either H/W 3D acceleration (if available) or the included optimized 3D S/W rasterizer.

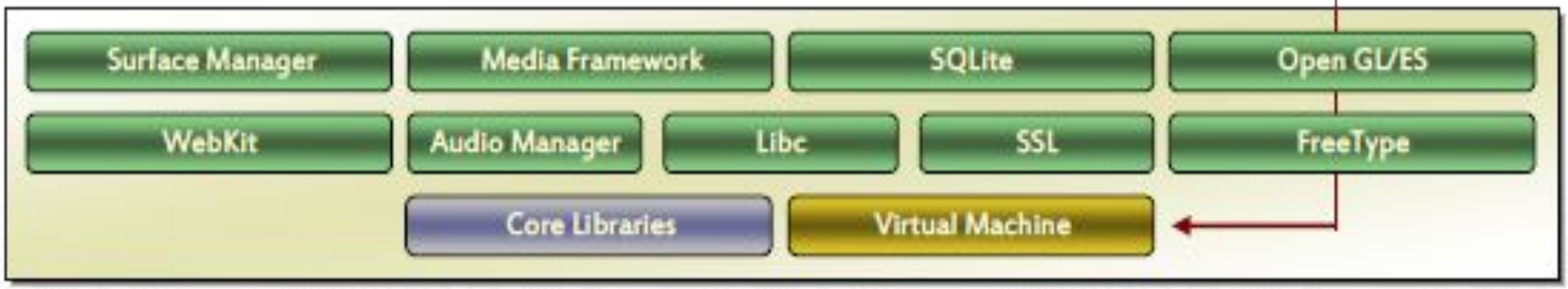
- **SGL -** SGL is Android's specialized 2D graphics library

Native Libraries – FreeType & SSL

- **FreeType**
  - Implémentas a font rastérisation engin
  - Written in C
  - It is used to rasterize characters into bitmaps and provides support for other font-related operations
- **SSL** – Secure Socket Layer for networking.

Android Runtime





Android Runtime – Overview

- A vital component in the Android stack
- Composed of two major parts:
  - The Dalvik virtual machine
  - Core libraries

Android Runtime – Dalvik Virtual Machine

- One of the most important components in Android
- Developed by Dan Bornstein, and named after the Dalvik village in Iceland
- Three main design targets:
  - runs on a limited CPU and RAM (250-500MHz)(20-40 MB)
  - runs atop an OS with no swap space
  - runs on a device with limited power

Android Runtime – Dalvik Virtual Machine

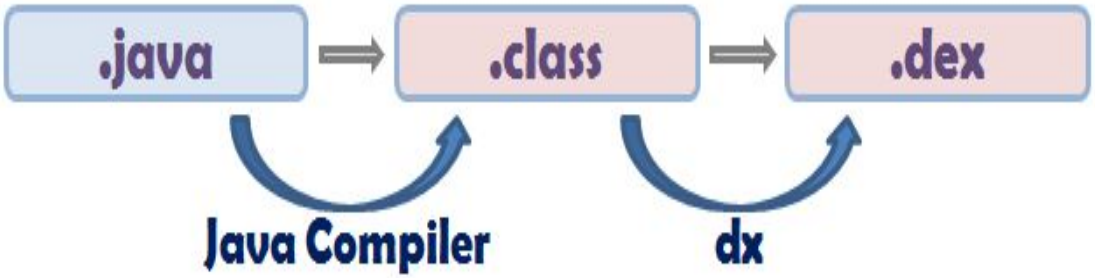
- Android's custom virtual machine based on Java VM
- Providing environment on which every Android application runs
  - Each Android application runs in its own process, with its own instance of the Dalvik VM.
  - Dalvik has been written so that a device can run multiple VMs efficiently.

Android Runtime – Dalvik Virtual Machine

That's important for few reasons:

- First, no application is depend upon another
- Second, If an application crashes, it shouldn't affect any other applications running on the device
- Third, it simplifies memory management

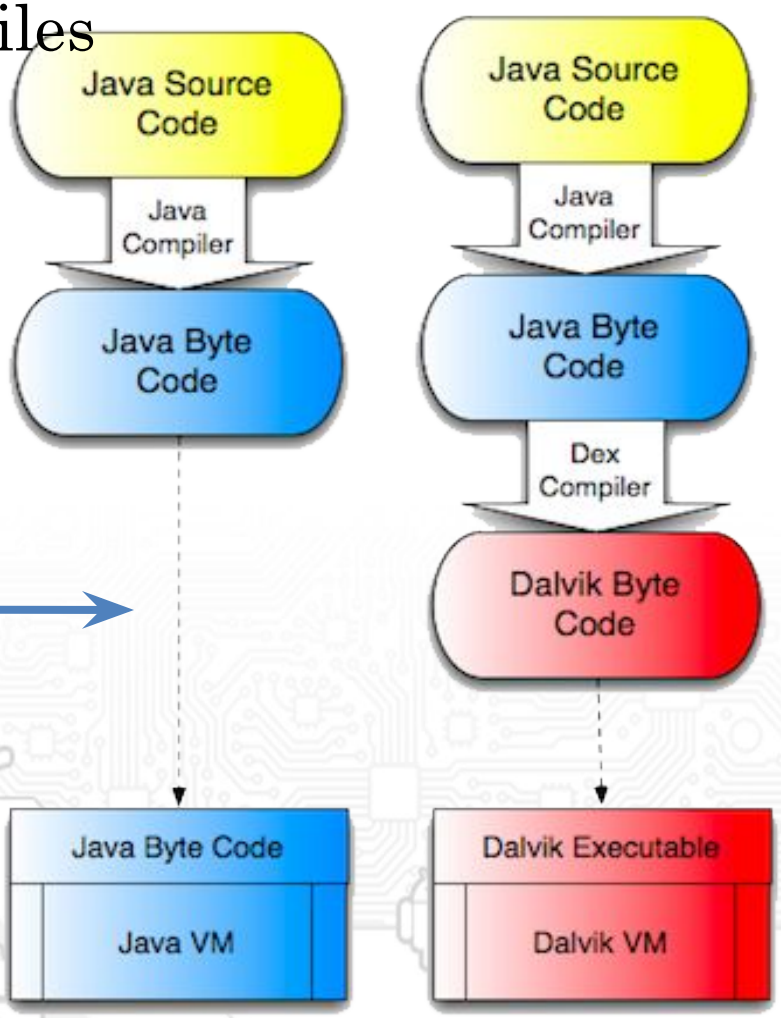
Dalvik Virtual Machine



- The VM runs Java applications which have been converted to the Dalvik Executable format .dex.
- Java Source code is compiled into *java bytecode* by java compiler, which is stored with in .class files
- The “dx” tool available in the SDK converts java bytecode into DVM bytecode at build time, which is a .dex or dalvik executable file.

Android Runtime – Dalvik Virtual Machine

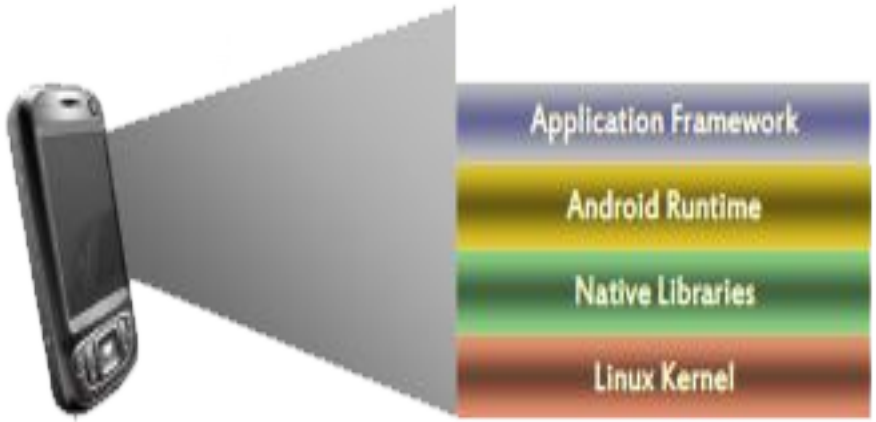
- Dalvik runs dex files, which are converted at compile time from standard class and jar files
- Dex files are more Compact and efficient than class files and is optimized for minimal memory footprint.



Android Runtime – Core Libraries

- Android's core libraries provide most of the functionality available in the core libraries of Java needed for applications development(I/O functionalities, Collections, File Access etc)
- This provide a powerful, yet simple and familiar development platform.
- APIs
  - Data Structures
  - Utilities
  - File Access
  - Network Access
  - Graphics
  - Etc ….

Application Framework



Application Framework

Before we move on to the application framework we must know:

- API - Application Programming Interface, An Interface implemented to enable interaction by this program and other software programs. (Similar to GUI which facilitates the interaction between user and this program)
- Application Development Framework - Consists of a software framework used by software developers to implement the standard structure of an application for a specific development environment (such as an operating system or a web application).
- Software Framework - A set of libraries which has been built for a specific platform.

Application Framework

- Services that are essential to the Android platform
  - Core Platform Services
  - Hardware Services
- API Interface, Enabling and simplifying the reuse of components
  - Developers have full access to the same framework APIs used by the core applications.
- Think of the application framework as a set of basic tools with which a developer can build much more complex tools.

Application Framework

Activity Manager

- Managing the lifecycle of applications and providing a common navigation backstack

Package Manager

- Holds information about applications loaded in the system

Window Manager

- The window manager creates display surfaces for the application
- Responsible for organizing the screen and display of different layers of application

Application Framework

Resource Manager

- Providing access to non-code resources (localized string, graphics, and layout les).

Content Providers

- Enabling applications to access data from other applications or to share their own data.

View System

- Used to build an application, including lists, grids, text boxes, buttons, and embedded web

Application Framework

Notification Manager

- Enabling all applications to display customer alerts in the status bar.

Telephony Manager

- Provides core telephoning functionalities.

Location Manager

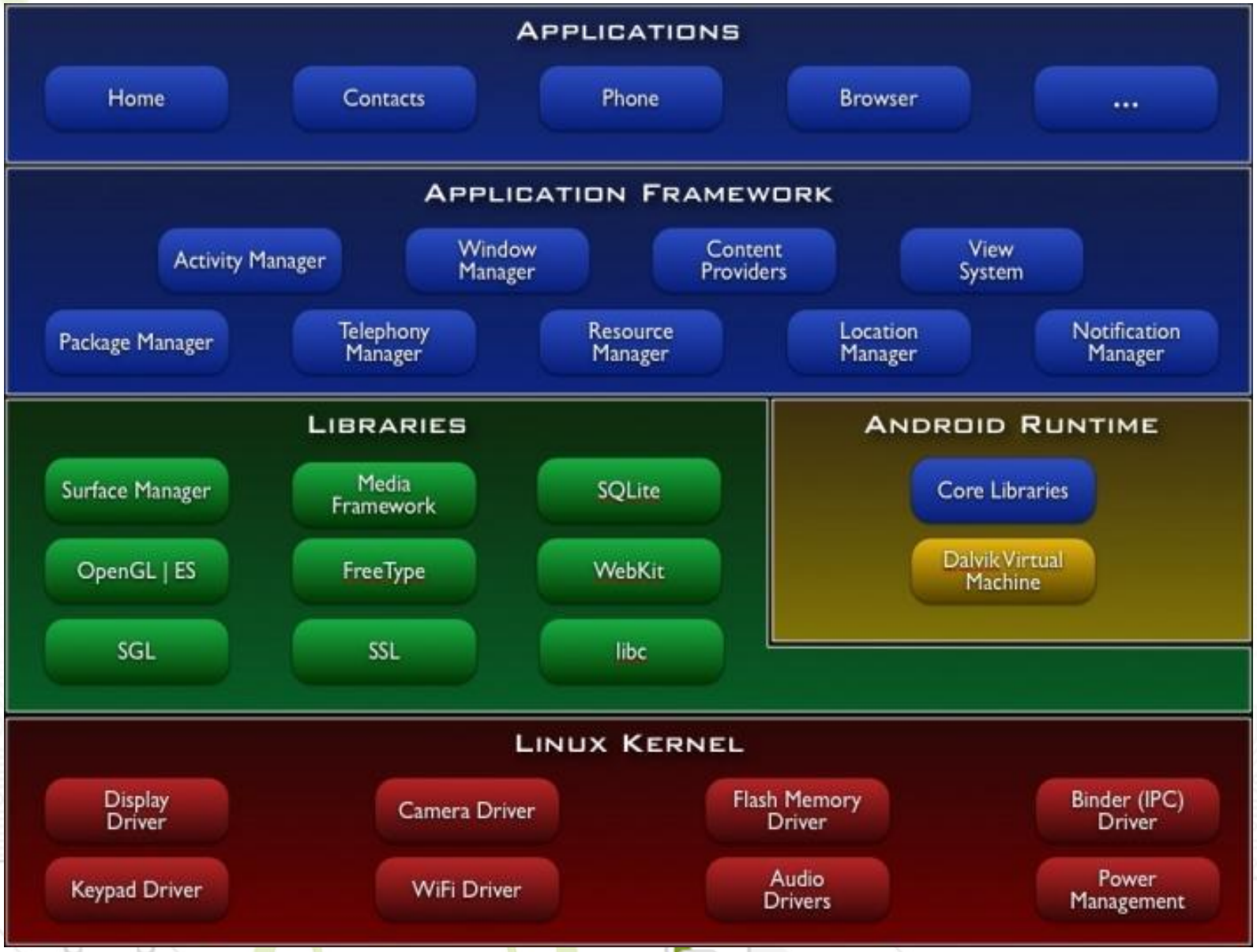
- Allows applications to obtain periodic updates
  - device's geographical location
  - Triggers an application-specified event when the device enters the proximity of a given geographical location.

Application Framework

XMPP Service

- An Open, XML-based protocol originally aimed at near-real-time, extensible instant messaging
- Largely extended. VoIP
- Aims to:
  - Send simple P2P message between handsets
  - Send and receive IM's using Google's Talk

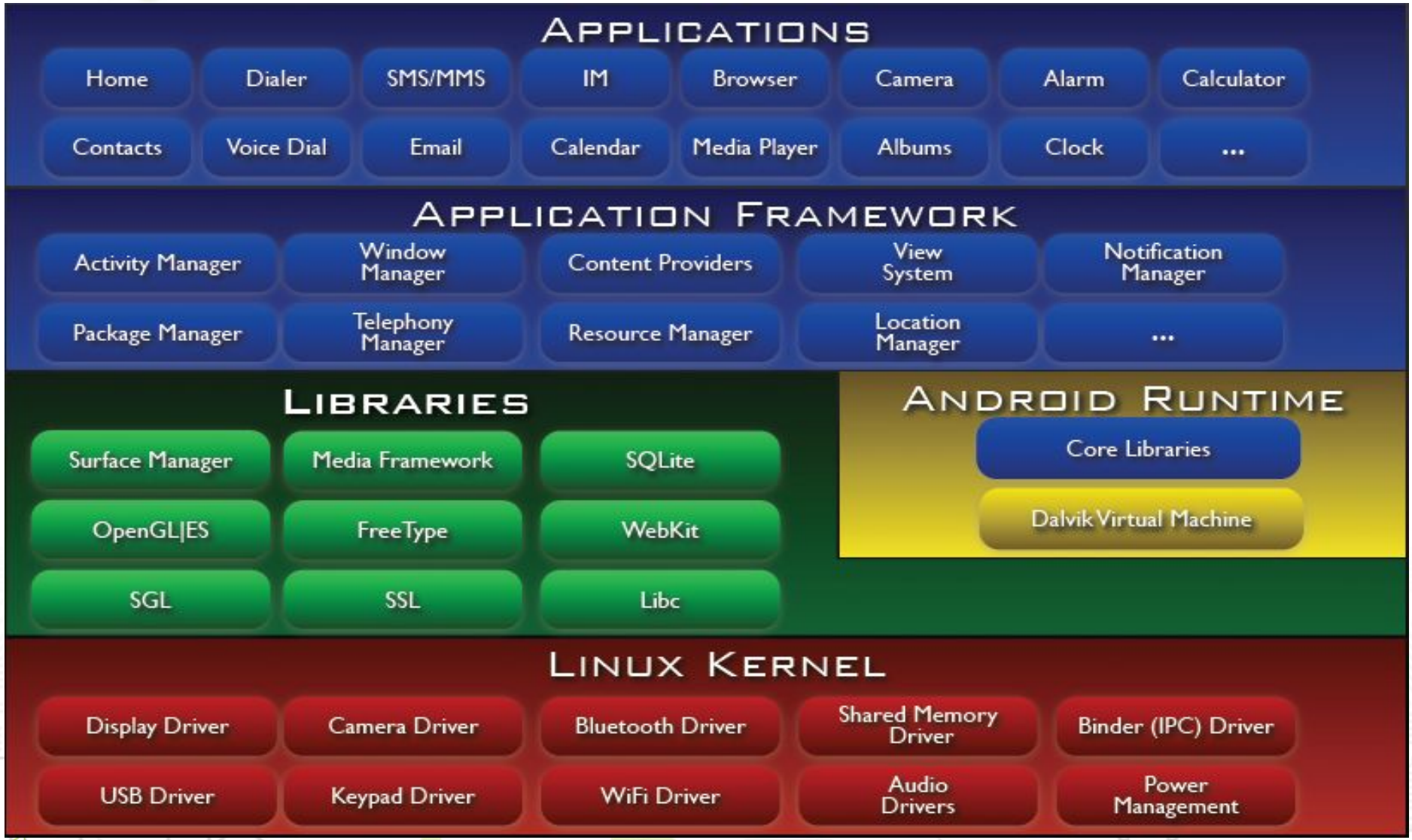
Applications

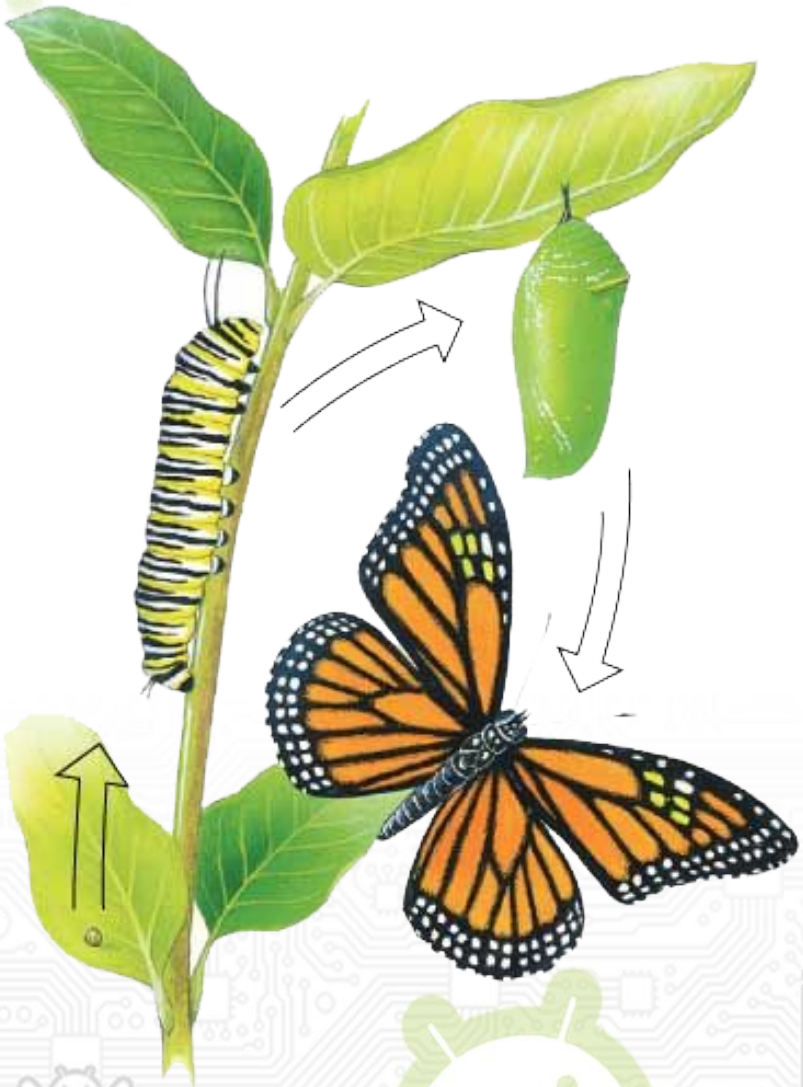


Applications

- Built in and user application
- Android provides a set of core applications:
  - Email Client ,SMS Program, Calendar
  - Maps, Browser, Contacts
  - Etc….
- All applications are written using the Java programming language.

Android Architecture





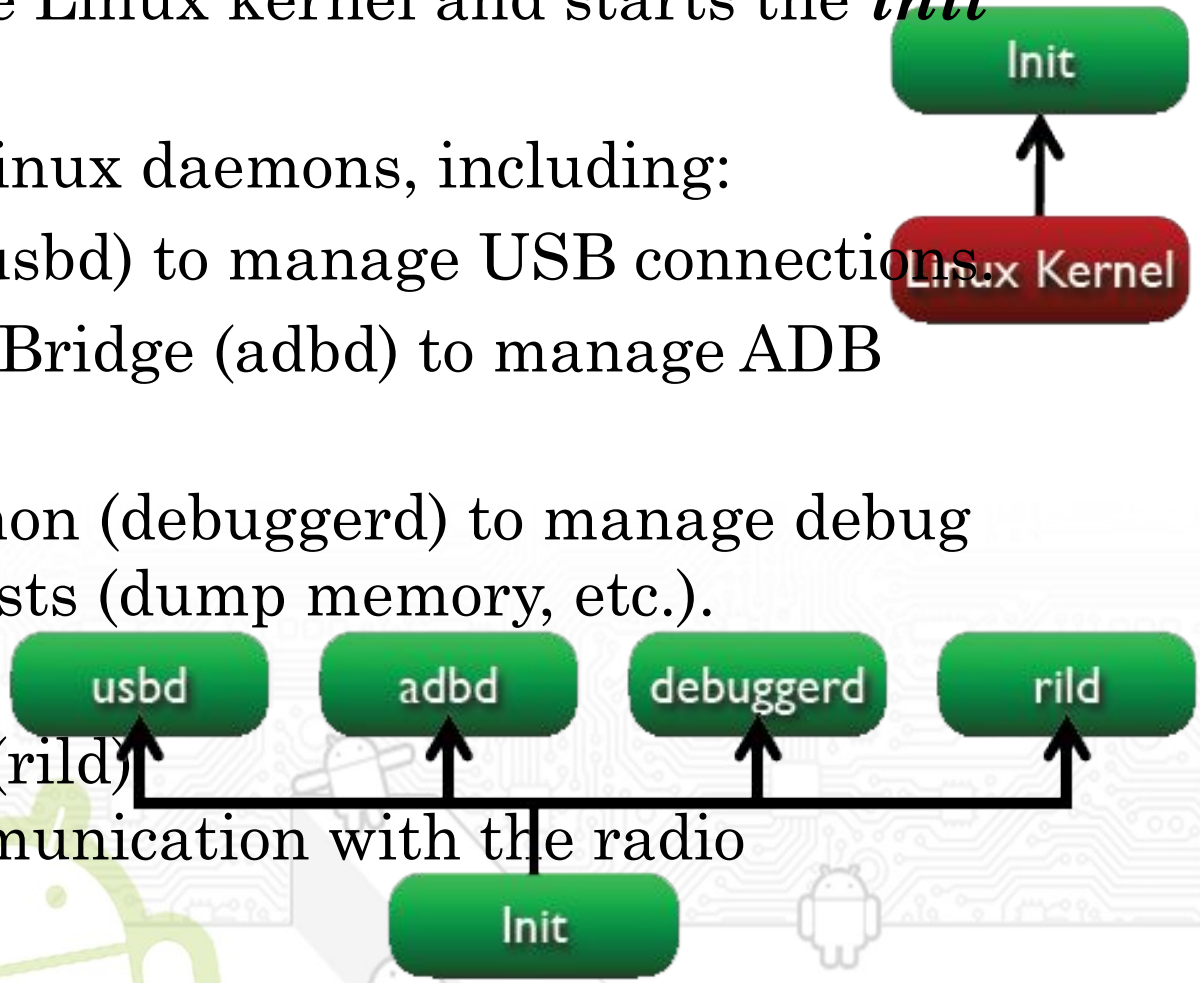
ANDROID START UP & RUN TIME

aitrich TECHNOLOGIES

aTP aitrich trained professional

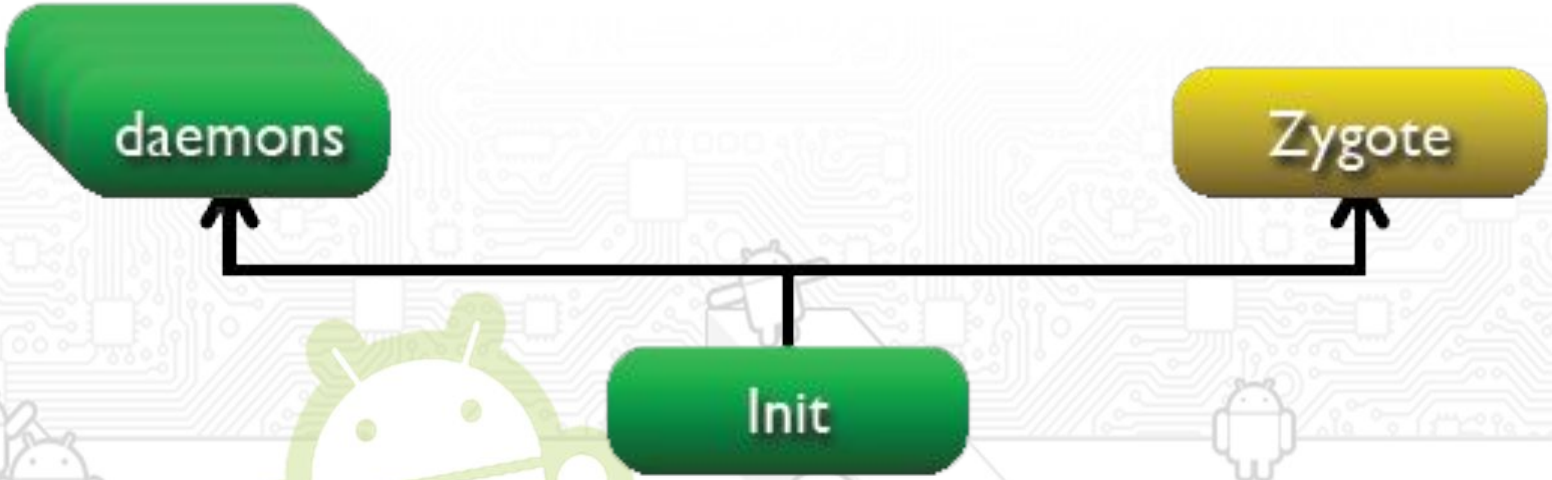
Platform Start-up

- Its all starts with init…
- Similar to most Linux-based systems at start-up, the ***bootloader*** loads the Linux kernel and starts the ***init*** process.
- **Init** starts various Linux daemons, including:
  - USB Daemon (usbd) to manage USB connections.
  - Android Debug Bridge (adbd) to manage ADB connections.
  - Debugger Daemon (debuggerd) to manage debug processes requests (dump memory, etc.).
  - Radio Interface Layer Daemon (rild) to manage communication with the radio



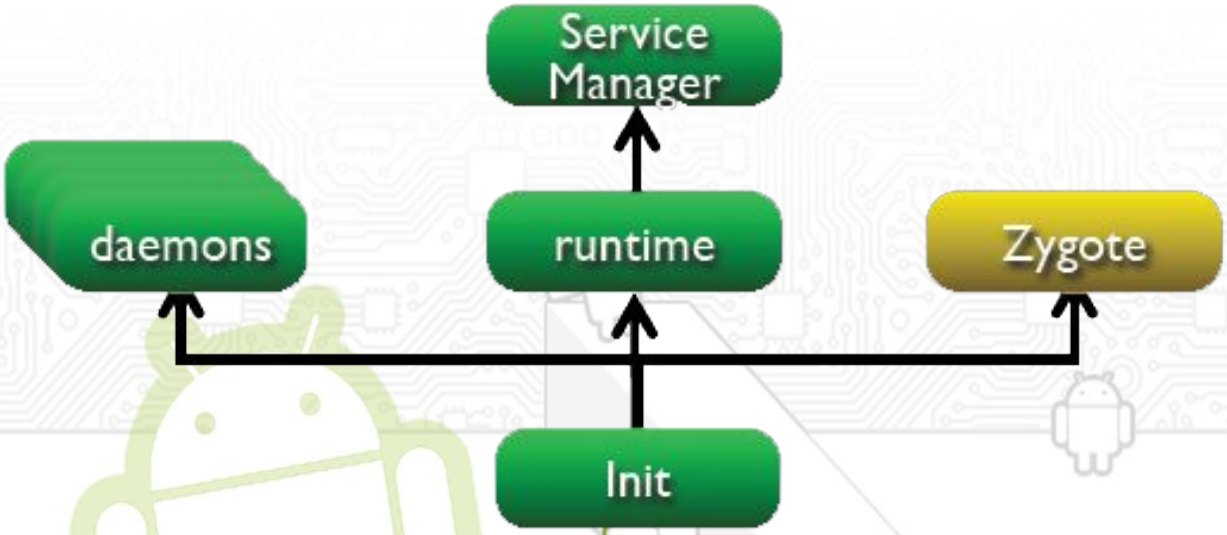
Platform Start-up

- Then Init start **Zygote** process
  - The **Zygote** process initializes a Dalvik VM instance and loads a bunch of libraries and starts listening on a socket for requests to spawn more processes (with VM instances).



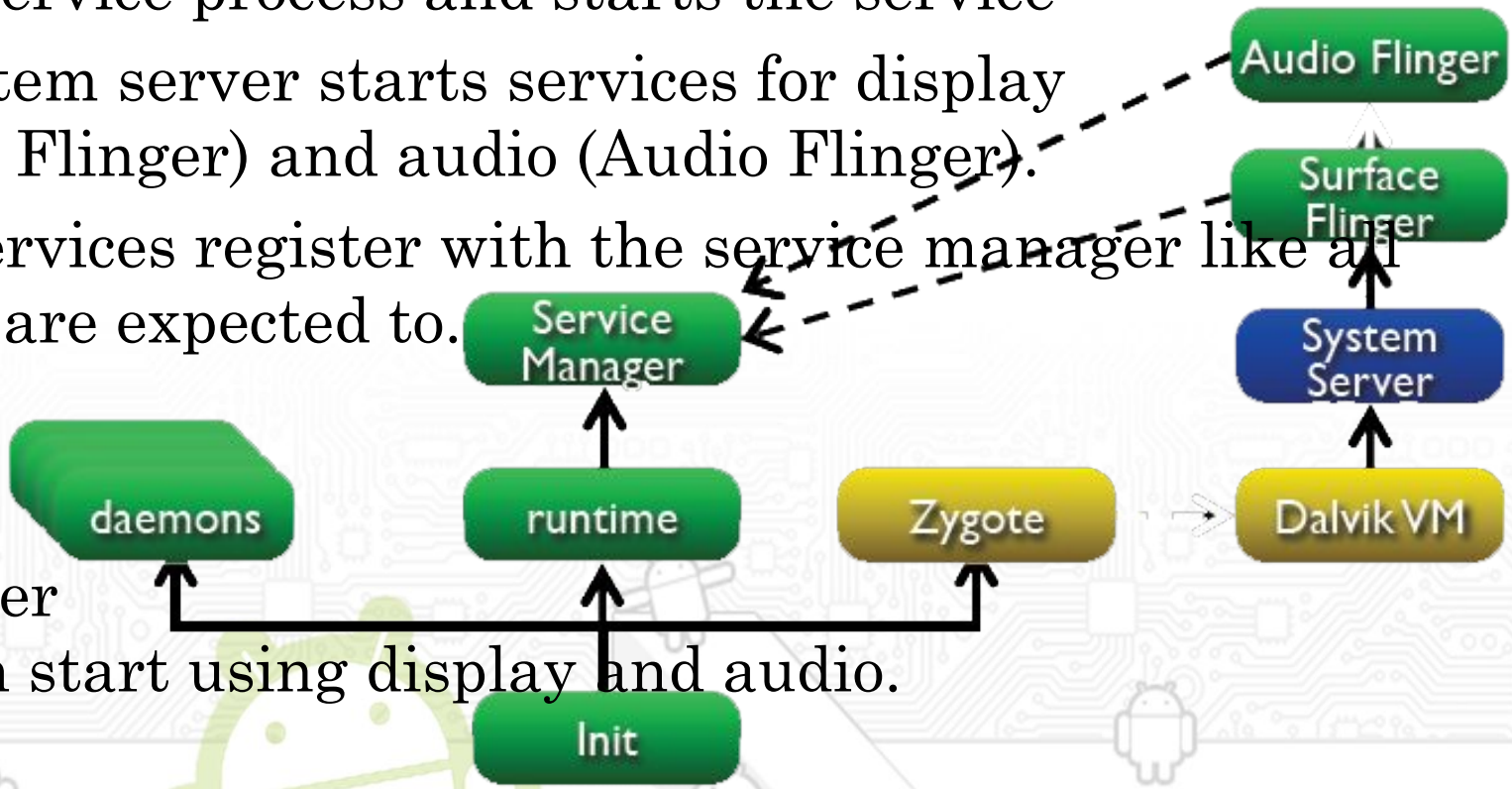
Platform Start-up

- Once Zygote is in place, the Init starts runtime process. This basically does two things:
  - 1. Initializes Service Manager - the context manager for Binder that handles service registration and lookup.
  - Registers Service Manager as default context manager for Binder services.



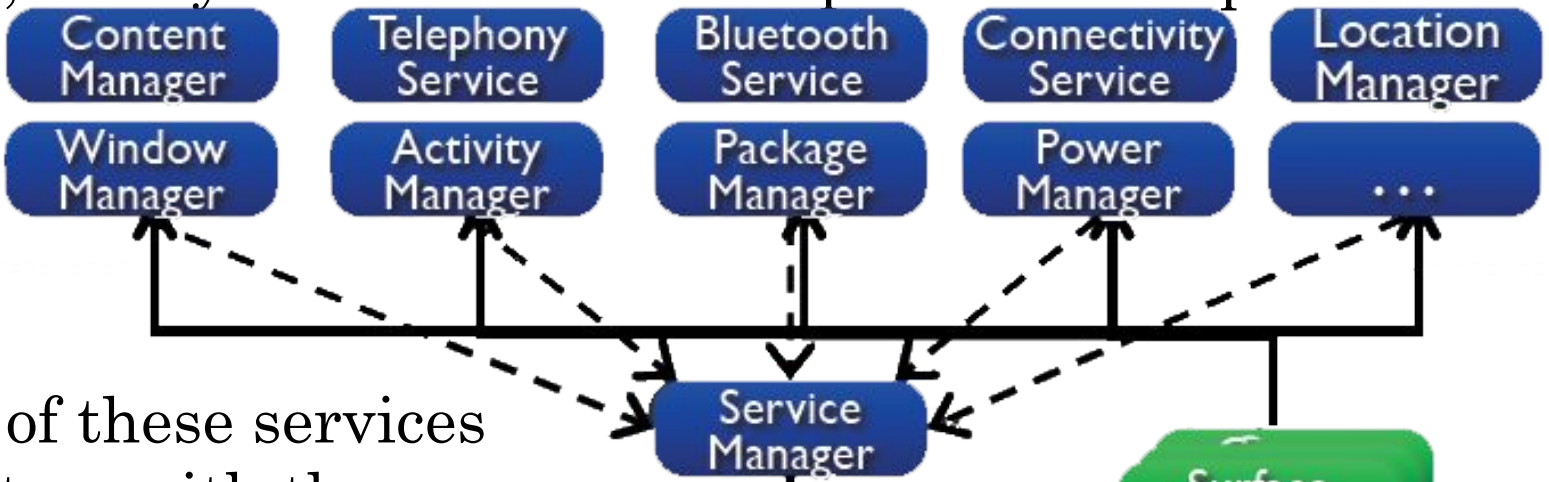
Platform Start-up

- **Runtime** process…
  - 2. Sends request for Zygote to start System Server.
  - Zygote forks a new VM instance for the System Service process and starts the service
- The System server starts services for display (Surface Flinger) and audio (Audio Flinger).
- These services register with the service manager like all services are expected to.
- Now other apps can start using display and audio.

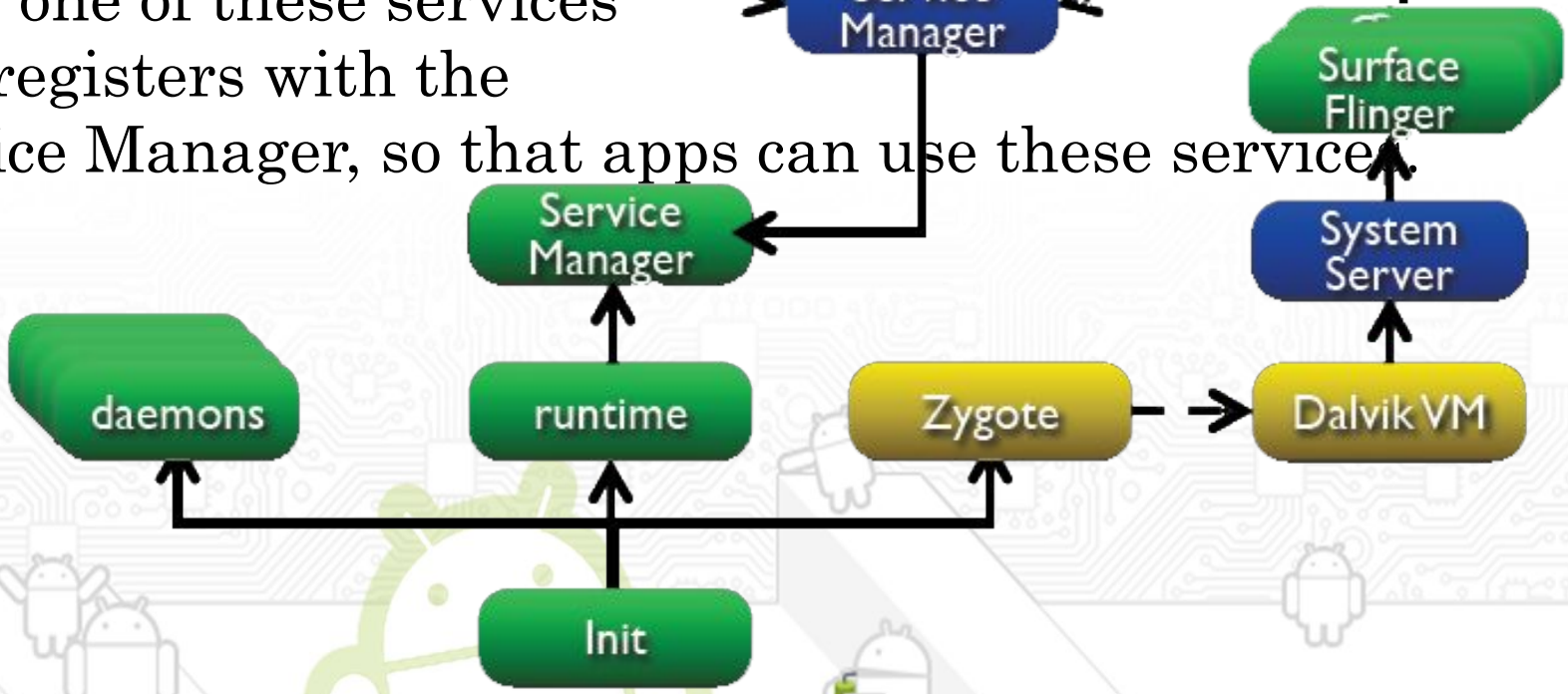


Platform Start-up

- After this, the System server starts up all the core platform services.

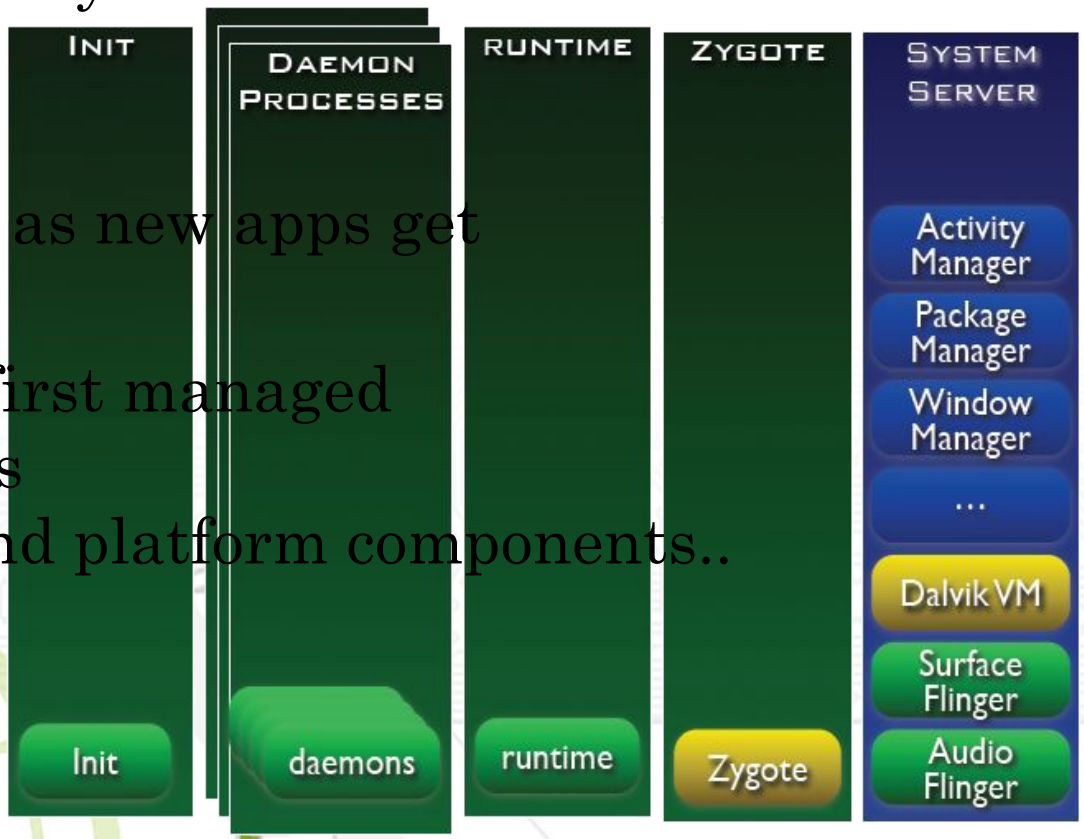


- Each one of these services also registers with the Service Manager, so that apps can use these services.



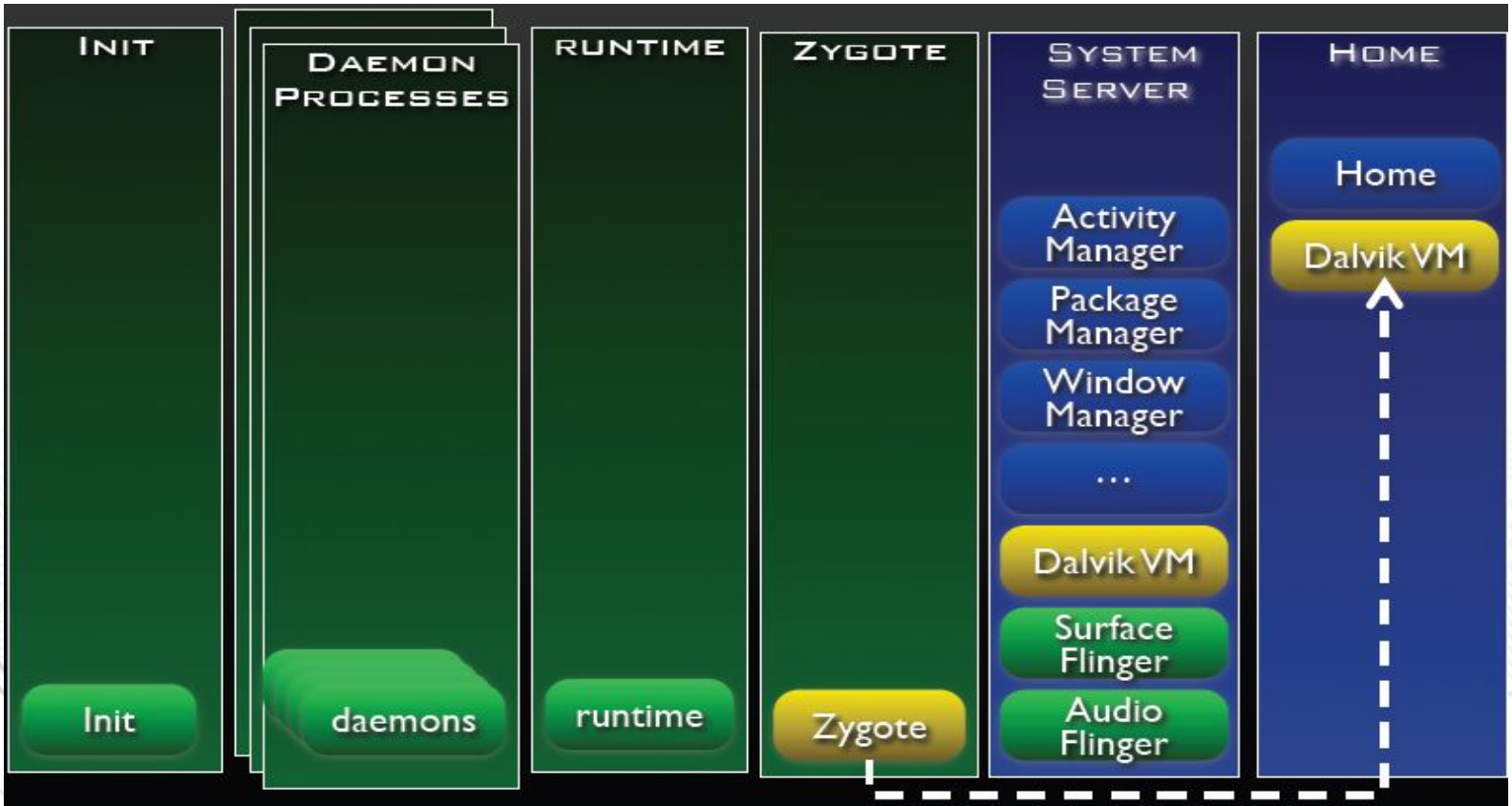
Platform Start-up

- After system server loads all services, the system is ready…
- At this stage we have the following processes in place:
  - Daemons – started by init
  - Runtime – also started by init
  - Zygote – the original zygote which will continually get forked as new apps get launched
  - System Server – the first managed process which contains all the core services and platform components..



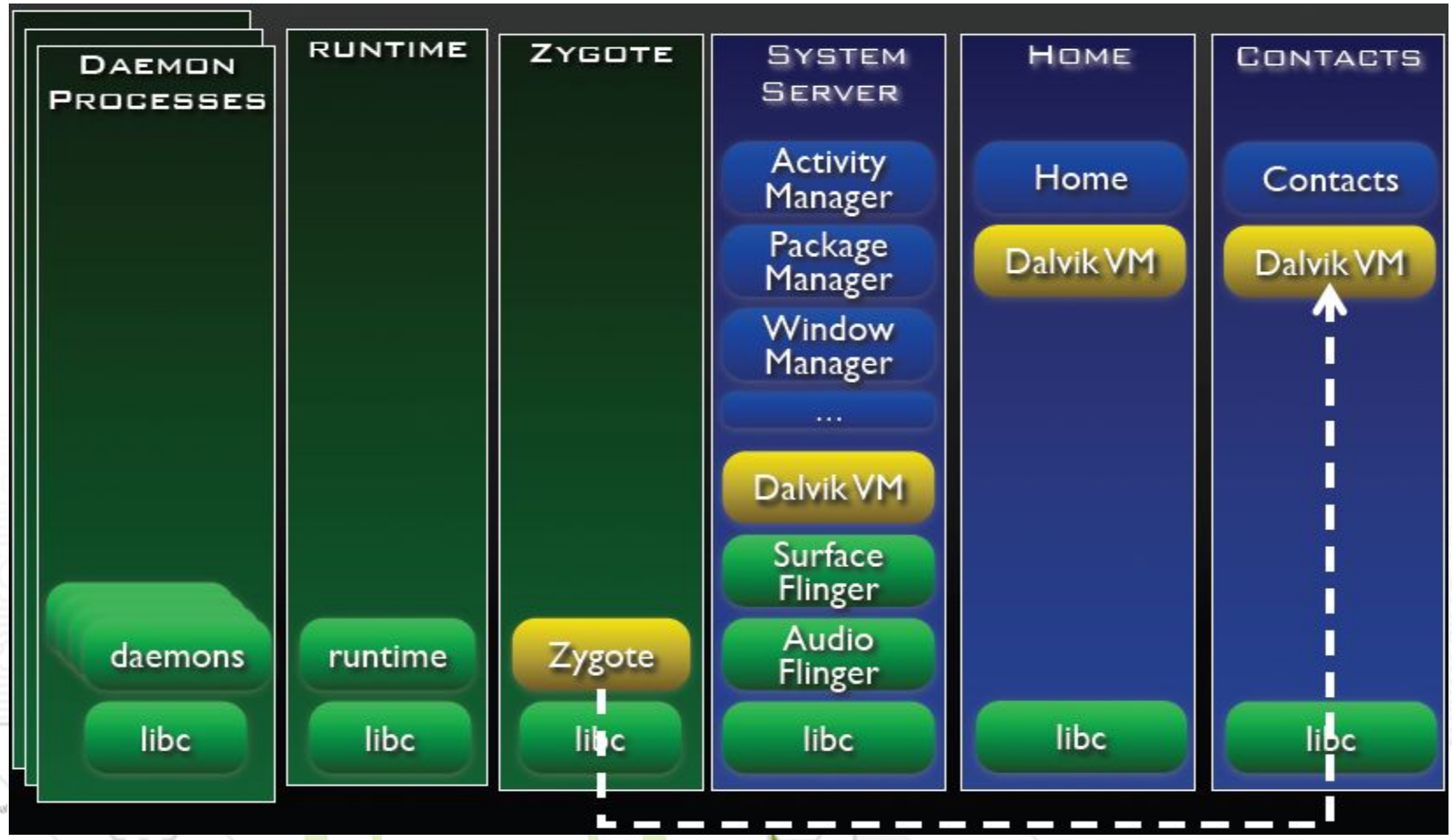
Runtime Overview

- After this the home screen or the Idle screen is launched
  - Basically the Activity Manager (which is inside the System Server) sends a message to Zygote to start the "Home" Activity, which causes Zygote to fork into a new process with a Dalvik VM and the Home activity.



Runtime Overview

- Each subsequent application is launched in it's own process. for example, if the user starts the Contacts app, the apt process would be setup:

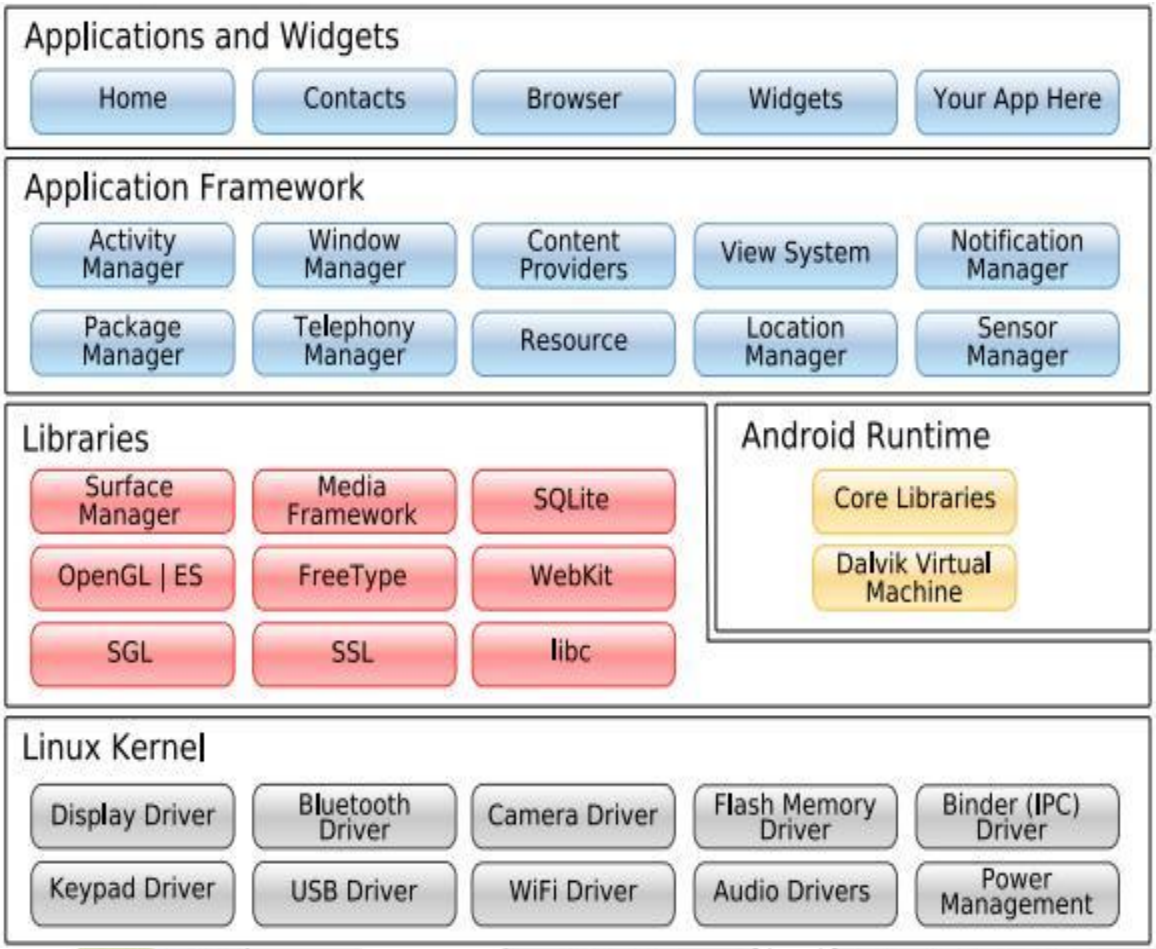


Layer Interaction

There are three main ways for your App to talk to native library

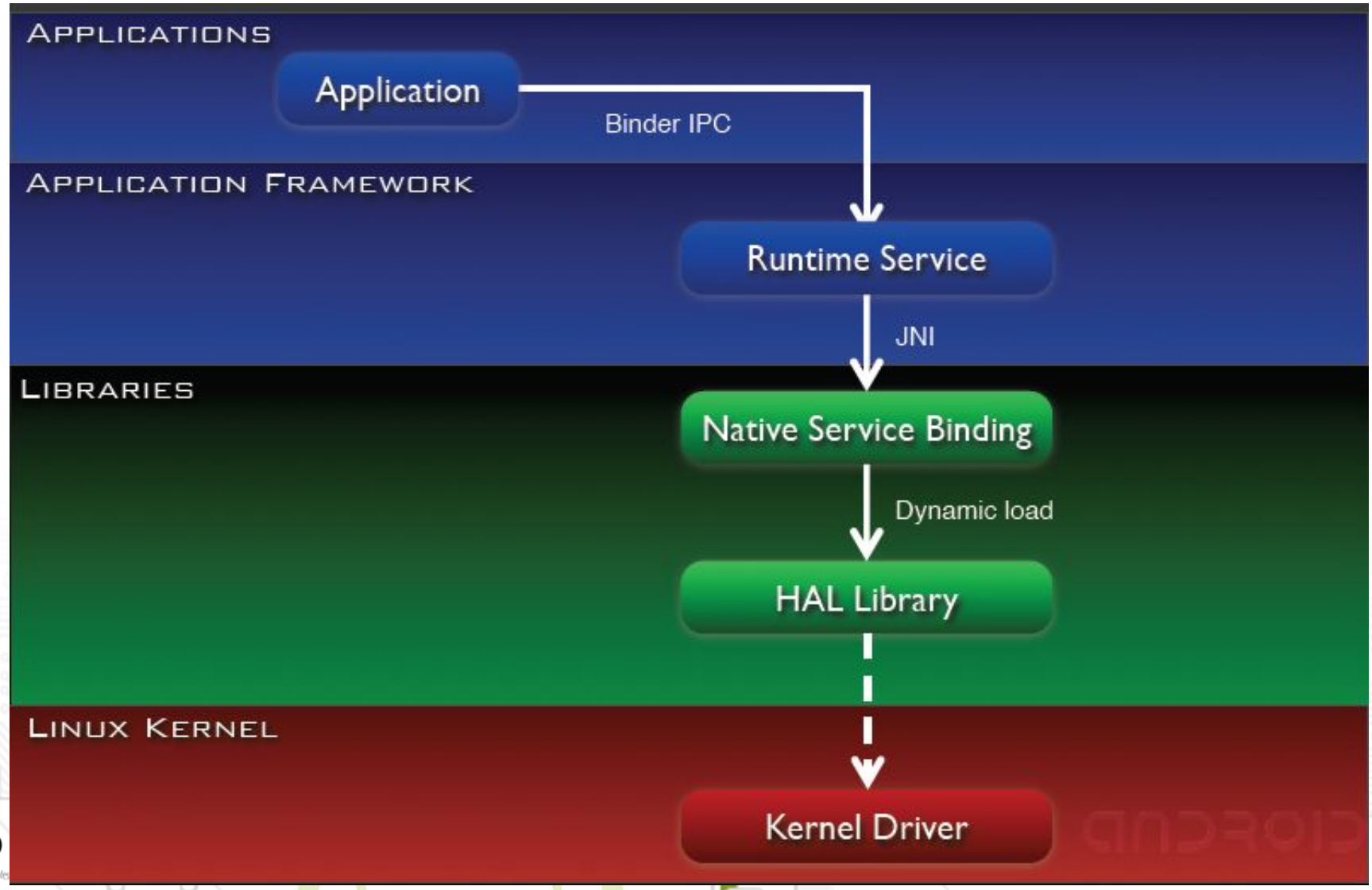
- Directly
- via Native Service
- via Native Daemon

- It will depend on the type of app and type of native library which method works best



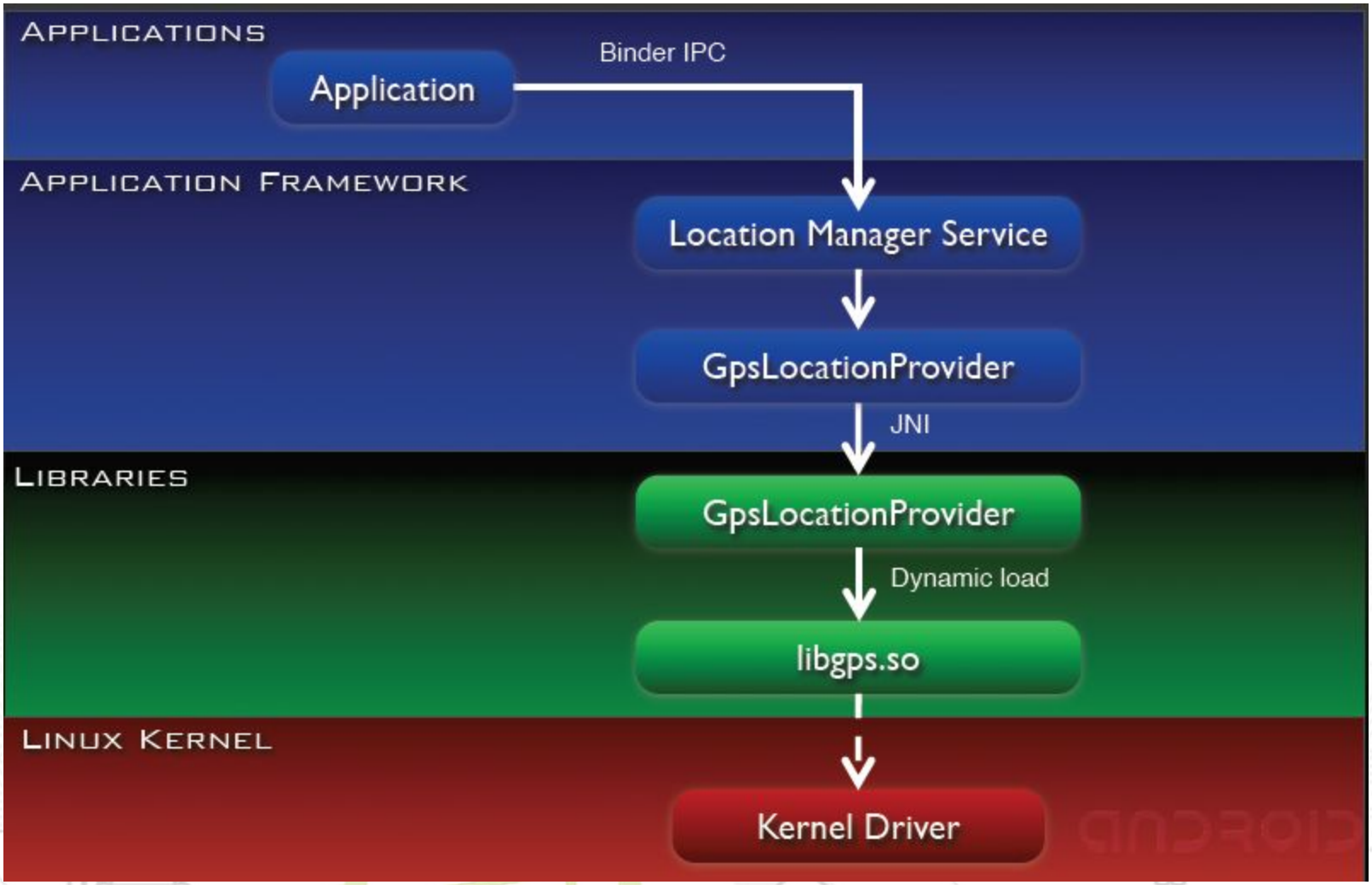
Layer Interaction

1. App -> Runtime Service -> lib



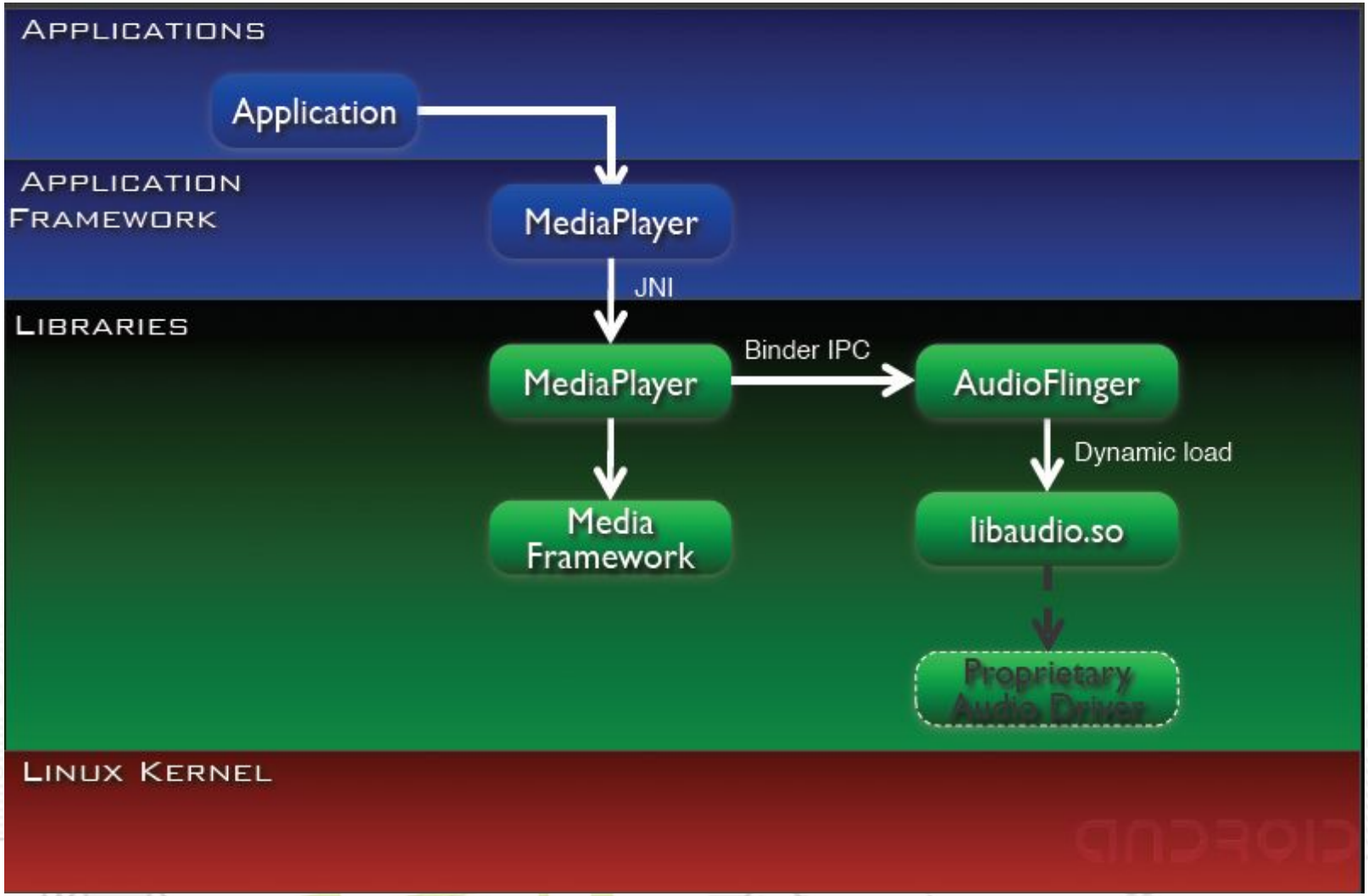
Layer Interaction

Example: Location Manager



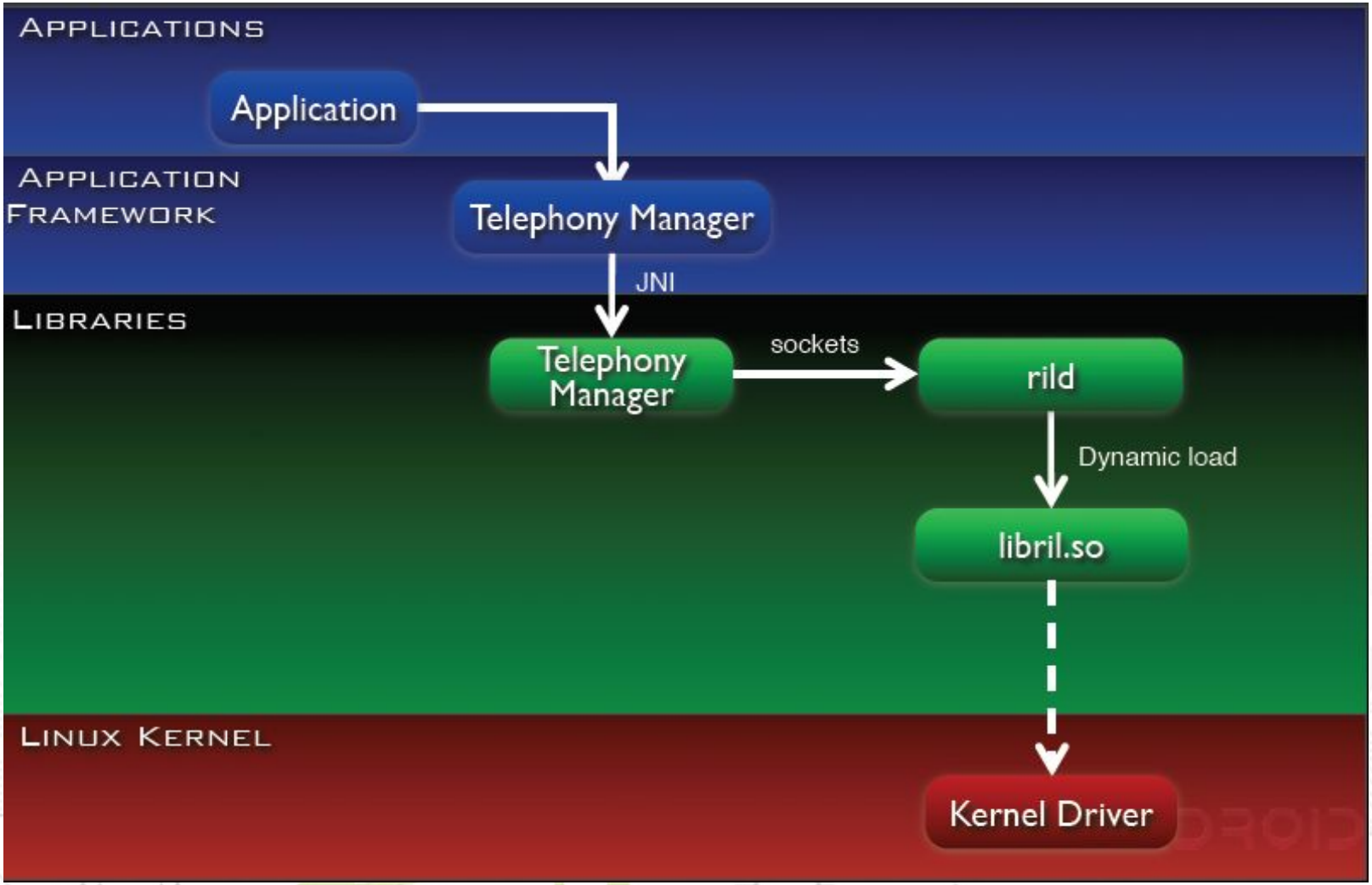
Layer Interaction

2. App -> Runtime Service -> Native Service -> lib



Layer Interaction

3. App -> Runtime Service -> Native Daemon -> lib



Layer Interaction

Three main flavours of Android layer cake:

- 1. App -> Runtime Service -> lib
- 2. App -> Runtime Service -> Native Service -> lib
- 3. App -> Runtime Service -> Native Daemon -> lib

References

- http://developer.android.com/guide/basics/what-is-android.html
- https://sites.google.com/site/io/dalvik-vm-internals
- http://www.brighthub.com/mobile/google-android/articles/17822.aspx
- http://sites.google.com/site/io/anatomy--physiology-of-an-android